

-----#This program says hello and asks for my name-----

```
print('Hello World')
print('What is your name?')
myName = input()
print('It is good to meet you, '+myName+'!')
print('The length of your name is : '+str(len(myName))) #len() fn used here
print('What is your age, '+myName+' ?')
myAge = input()
print('You will be ' + str(int(myAge)+1) + ' in a year.')
```

---#Conditions IF-ELIF-ELSE-----

```
name1 = 'Alice'
print(name1)
if name1 == 'ALICE': print(name1); print('Hello '+name1+'!')
else: print('Wrong Name!'); print('Who are you?')

print('Enter your name: ')
name2= input()
if name2 == 'Alice': print('Hi '+name2+'!')
else: print('Hi '+name2+'!')

print('Enter your number: ')
num1= int(input()) #typecasting string input of user to integer
if num1 >= 33: print('Good')
elif 10<=num1<=32: print('Hopeless!')
else: print('Just Die!')
```

----#Functions IN Python-----

```
def hello(): print('Hey Bro!')
hello()
print('Out of HELLO fn.')

def param(userInput):
    print('You have entered: '+str(userInput))
    userInput+=1
    print('Number is now: '+str(userInput))
print('Enter number: ')
num = int(input())
param(num)
```

-----#The 'return' keyword-----

```
import random
def getAnswer(answerNumber):
    if answerNumber == 1: return 'You are number one!'
    elif answerNumber == 2: return 'Second in Command!'
    elif answerNumber == 3: return 'Try Again!'
r = random.randint(1,3) #outside this range will print 'None'
result = getAnswer(r) #print(getAnswer(random.randint(1,3))) will give same result
print(result)
```

-----# 'While' & 'For' LOOPS-----

```
spam=0
while spam<5:
    print('Hello World.')
    spam+=1

name=''
while name!='Goku':
    print('Please type your name: ')
    name=input()
print('Thank you!')

while True:
    print('This is demo for BREAK. Enter name: ')
    name = input()
    if name == 'SARVESH': break
```

```

print('Thanks!')

while True:
    print('This is CONTINUE. Enter name: ')
    name = input()
    if name!='ALPHA': continue
    print('Hi '+name+'! Which city?')
    city = input()
    if city == 'Pune': break
print('Welcome to '+city+', '+name+'.')

while True:
    print('Who are you?')
    name = input()
    if name == 'Joe':
        print('Hello '+name+'! What is the password? (It is a fish.)')
        password = input()
        if password == 'swordfish': print('Access Granted.');
```

 break
 else: continue
else: continue

print('My name is ')
for i in range(5): print('Jimmy five times ('+str(i)+'))'

total=0
for number in range(101): total+=number
print(total)
print('-----')

for i in range(12,16): print(i)
print('-----')

count = 0
for n in range(0,10,2): print(n); count+=1
print('The count is: '+str(count))
for i in range (5,-1,-1): print(i)

-----#random module in Python-----
import random
print('Here are 3 random numbers as specified in range of for-loop:')
for i in range(3): print(random.randint(1,10)) #3 nos. from 1 to 10

-----#Demo of 'sys' module in Python-----
import sys
while not False:
 print('Demo of sys module. Type exit to exit.')
 response = input()
 if response == 'exit' or response == 'Exit' or response == 'EXIT': sys.exit()
 print('You typed: '+response+'.')

-----#This Is Ration Bill Calculator-----
totalBill = 0
while True:
 inputPrice = input('Enter MRP of item:')
 if inputPrice!='q':
 totalBill = totalBill + float(inputPrice)
 print(f'Your bill so far: INR {totalBill}')
 else:
 print(f'Total bill is: INR {totalBill}')
 print('-----')
 print('Checking out.')
 break

-----#Local & Global Scopes-----
def spam():
 eggs = 99
 bacon()
 print(eggs)

```

def bacon():
    ham = 101
    eggs = 0

spam() #this will print 99 as 'eggs' of bacon() is local to it

def spam(): print(eggs)

eggs=42 #this global variable is accessible in spam()
spam()
print(eggs) #prints 42

def spam():
    eggs='Spam Local'
    print(eggs) #prints 'Spam Local'
def bacon():
    eggs='Bacon Local'
    print(eggs) #prints 'Bacon Local'
    spam()
    print(eggs) #still prints 'Bacon Local'
eggs='Global'
bacon()
print(eggs) #prints 'Global'

def spam():
    global eggs #global keyword changes scope of 'eggs' below
    eggs='spam'
eggs = 'global'
spam()
print(eggs) #prints 'spam' coz fn has declared 'eggs' as global

def spam():
    print(eggs) #generates error as 'eggs' not defined earlier
    eggs='Spam Local'
eggs='Global'
spam()

```

-----Exception Handling-----

```

def spam(divideBy): return 42/divideBy
print(spam(2)) #prints result
print(spam(12)) #prints result
print(spam(0)) #error thrown & program terminated here
print(spam(1))

def spam(divideBy):
    try: return 42/divideBy
    except ZeroDivisionError: print('INVALID ARG PASSED!') #the divide by zero error

print(spam(2))
print(spam(12))
print(spam(0)) #prints 'except' block first, then prints 'None' as well
print(spam(1))

def spam(divideBy): return 42/divideBy
try:
    print(spam(2))
    print(spam(12))
    print(spam(0)) #program stops here after executing 'except' block
    print(spam(1)) #not executed as control doesn't return from 'except' block
except ZeroDivisionError: print('INVALID ARG PASSED!') #once here, control won't go back to 'try' block

```

-----Guess The Number-----

```

import random
secretNum = random.randint(1,20)
print('I\'m thinking of a number between 1 and 20...')

#Ask the player to guess 6 times.
for guessesTaken in range(1,7):
    print('Take a guess:')

```

```

guess = int(input())

if guess<secretNum: print('Your guess is too low.')
elif guess>secretNum: print('Your guess is too high.')
else: break
if guess==secretNum: print('Great! You guessed it right in '+str(guessesTaken)+' guesses!')
else: print('Nope! The number I was thinking of was: '+str(secretNum))

```

-----COLLATZ SEQUENCE-----

```

def collatz(number):
    hop=0 #to count number of hops or steps taken to reach 1
    while number>1:
        print(number, end=' ')
        if number%2==0: number=number//2; hop+=1
        else: number=3*number+1; hop+=1
    print(1, '\n', 'It took '+str(hop)+' hops to reach 1.')

```

```

#userInt = int(input('Input an integer: '))
print('Input an integer:')
userInt= int(input())
print('Collatz sequence: ')
collatz(userInt)

```

-----#FACTORIAL PROGRAM (WITH NUMBER OF TRAILING ZEROS IN FAC VALUE)---

```

def factorial(number):
    if number == 0 or number == 1: return 1
    else: return number * factorial(number-1)
def facTrailZero(number):
    count=0
    fac = factorial(number)
    while fac%10==0: count=count+1; fac=fac//10
    return count
def trailZero(number):
    count=0
    i=5
    while(number//i!=0): count = count+int(number/i); i=i*5
    return count

```

```

userInput = int(input("Enter a number:\n"))
print(f'Factorial of {userInput} is:', factorial(userInput))
print(f'Trailing zeroes in factorial of {userInput} are:', facTrailZero(userInput))
print(f'No. of 0s by trailZero method in {userInput} are:', trailZero(userInput))

```

-----#STRINGS OPERATIONS & FUNCTIONS-----

```

##[0:n] Reads first char at index 0 & upto char at 'n-1' pos
'''[0:n:m] Reads from first char at index 0, skipping every (m-1)th char till (n-1)th char; 'm' is called SLICE STEP'''
'''[n:0:-m] Reads from right to left; 'n'th char to 0+1th char skipping (-m+1) chars'''

```

```

'''FUNCTIONS: upper, lower, title, isupper, islower, isdigit, istitle, isdecimal, isalnum, isalpha,
isnumeric,endswith, count, find, capitalize, split, join, rjust, ljust, center, strip,rstrip, lstrip,
copy & paste (from 'pyperclip' module) '''

```

```

mystr = 'Harry is a good boy'

```

```

##print(mystr)
##print(len(mystr)) #O/P: 19
##print(mystr[4]) #O/P: 'y'
##print(mystr[5]) #O/P: single space
##print(mystr[0:4]) #O/P: 'Harr'
##print(mystr[0:19])
##print(mystr[78]) #Error: string index out of range
##print(mystr[0:78]) #same error as prev print

```

```

##print(mystr[0:7]) #O/P: 'Harry i'
##print(mystr[0:]) #O/P: entire mystr
##print(mystr[:-1]) #O/P: 'Harry is a good bo'
##print(mystr[:]) #O/P: entire mystr

```

```

##print(mystr[:]) #O/P: same as prev print
##print(mystr[:7]) #O/P: 'Harry i'

##print(mystr[12:1]) #prints nothing; see next 'print',
##print(mystr[12:1:-1]) #O/P: 'og a si yrr'
##print(mystr[12:0:-1]) #O/P: 'og a si yrra'
##print(mystr[12::-1]) #O/P: 'og a si yrraH'
##print(mystr[:-1]) #O/P: 'yob doog a si yrraH'
##print(mystr[:-2]) #O/P: 'ybdo iyrH'
##print(mystr[12:]) #O/P: 'ood boy'
##print(mystr[12::]) #O/P: same as above

##print(mystr[12:0:0]) #Error: Slice step can't be 0
##print(mystr[14:1:-1]) #O/P: 'doog a si yrr'
##print(mystr[:50]) #O/P: 'H' for 50 or any out of bound slice-step value
##print(mystr[-6:]) #O/P: 'od boy'
##print(mystr[-6:0]) #prints nothing for 0,1,2 ...n
##print(mystr[-6:-1:]) #O/P: 'od bo'; -1 means leave last 1 char of string

mystr1='thequickbrownfoxjumpedoverlazydog'
##print(mystr1[-13:-2:2]) #O/P: 'eozld'
##print(mystr1[-20:-3:-3]) #prints nothing due to -ve slice-step
##print(mystr1[-20:-25:]) #prints nothing
##print(mystr1[-20:-15:]) #O/P: 'foxju'
##print(mystr1[-4:-2]) #O/P: 'yd'

##print(mystr.isalnum()) #prints 'FALSE'
##print(mystr1.isalnum()) #prints 'TRUE'; 'FALSE' if space inserted in mystr1
##print(mystr1.isalpha()) #same as above o/p
##print(mystr1.endswith('god')) #O/P: False
##print(mystr1.count('b')) #O/P: 1
##print(mystr1.capitalize()) #O/P: Thequickbrownfoxjumpedoverlazydog
##print(mystr1.find('fox')) #returns char index of 'f' in 'fox'
##print(mystr1.upper()) #O/P: THEQUICKBROWNFOXJUMPEDOVLAZYDOG
##print('APPLE'.lower()) #O/P: apple

##print(mystr1.replace('fox','deer')) '''replaces 'fox' with 'deer' and prints it as such on screen;
no change in actual mystr1'''

mystr2 = 'The Count Of Monte Cristo' #length=25
mystr3 = 'THE COUNT OF MONTE CRISTO'
mystr4 = 'the count of monte cristo'
##print(mystr2.casefold()) #o/p: the count of monte cristo
##print(mystr3.casefold()) #o/p: same as above
##print(mystr4.casefold()) #o/p: same as above
##print(mystr4.casefold('COUNT')) #ERROR; 'casefold()' takes no args
##print(mystr2.center(26,'#')) ''' Length of mystr2 is 25. The 'center()' fn mystr2 with a # on extreme RIGHT of
mystr2 to make length 26.'''

##print(mystr2.center(27,'#')) '''In this print stmt, 'mystr2' length (which is 25) is padded by '#' FIRST ON THE
RIGHT end & then on LEFT end making length as 27'''

##print(mystr2.center(28,'#')) '''Here, 'mystr2' is padded with 2 hash symbols on the RIGHT FIRST and then one
hash symbol on the LEFT of mystr2, to make mystr2 length as 28'''

##print(mystr2.center(29,'#'))
##print(mystr2.center(29,'*')) #o/p: **The Count of Monte Cristo**
##print(mystr2.center(29)) #o/p: ' The Count of Monte Cristo '
##print(mystr4.isdecimal()) #returns FALSE
##print(mystr4.isidentifier()) #returns FALSE
##print(mystr4.isdigit()) #returns FALSE
##print(mystr4.isspace()) #returns FALSE
##print(mystr4.isprintable()) #returns TRUE
##print(mystr2.istitle()) #returns TRUE; first char of each word is capital

##print('The string is: %s'%(mystr2)) #prints 'mystr2'
##print('w' in mystr2) #returns FALSE
##print('ount' not in mystr2) #returns FALSE
##print (r'E://Scripts') #o/p: 'E://Scripts'

```

```

mystr5 = 'Java'
mystr6 = 'Python'
#print('{} and {} are programming langg.'.format(mystr5,mystr6))

#print(mystr2.split())#o/p: ['The', 'Count', 'Of', 'Monte', 'Cristo']
#print(mystr2.split('t'))#o/p: ['The Coun', ' Of Mon', 'e Cris', 'o'] '''split at all 't' '''

#print(mystr2.split('t',1))#o/p: ['The Coun', ' Of Monte Cristo'] '''split at first 't' '''

#print(mystr2.split('Count'))#o/p:['The ', ' Of Monte Cristo']
#print(mystr2.rstrip('Cristo')) #o/p:The Count Of Monte
#print(mystr2.lstrip('The')) #o/p: Count of Monte Cristo
#print(mystr2.swapcase()) #o/p: tHE cOUNT oF mONTE cRISTO
#print(mystr2.partition('Of')) #o/p: ('The Count ', 'Of', ' Monte Cristo')

'''Breaking a string into list items & storing those list items in a dictionary'''
#string = 'Hello'
#mylist=[]
##for i in string: mylist.append(i)
#print('The list of string items is: \n'+str(mylist))
#print('The popped out items are: ')
#while True:
#    print(mylist.pop())
#    if mylist==[]: break
#    else: continue
#mydict={}
#keys=['key1','key2','key3','key4','key5']
##for i in range(0,len(mylist)): mydict[keys[i]]=mylist[i]
#print('Dictionary of string: \n',mydict) #O/P: Dictionary of string:{'key1': 'H', 'key2': 'e', 'key3': 'l',
#    'key4': 'l', 'key5': 'o'}

```

'''Sweigart P-90: PicnicTable Prog'''

```

def printPicnic(itemsDict, leftWidth, rightWidth):
    print('PICNIC ITEMS'.center(leftWidth + rightWidth,'-'))
    for k,v in itemsDict.items(): print(k.ljust(leftWidth, '.') + str(v).rjust(rightWidth))
picnicItems = {'Sandwiches':4, 'Apples':12, 'Cups':4, 'Cookies':80}
printPicnic(picnicItems,12,5)
printPicnic(picnicItems,20,6)

```

'''OUTPUT:

```

---PICNIC ITEMS--
Sandwiches..    4
Apples.....   12
Cups.....      4
Cookies.....   80
-----PICNIC ITEMS-----
Sandwiches.....    4
Apples.....        12
Cups.....          4
Cookies.....       80
'''

```

'''Below 'print' stmt prints 'BaconSpamEggs' since strip() removes all instances of a,m,p and s from BOTH ends of string in 'spam' variable. The order of the characters in the string passed to strip() does not matter; strip('ampS') will do the same thing as strip('mapS') or strip('Spam').'''

```

spam='SpamSpamBaconSpamEggsSpamSpam'
print(spam.strip('ampS')) #O/P: BaconSpamEggs

```

-----LISTS-----

'''Lists are MUTABLE. (Tuples are immutable). List methods are: APPEND, CLEAR, COPY, COUNT, EXTEND, INDEX, INSERT, POP, REVERSE, REMOVE, SORT'''

```

grocery = ['Soap','Brush','Chair','Kite','Fish','Pen','Eraser']
numbers = [43,21,43,87,56,34,6,90,65,13,28]

```

```

grocery.sort
print(type(grocery)) #returns 'list' class type

```

```

print(type(numbers)) #returns 'list' class type
print(grocery)
print(numbers)
print(grocery[1]) #o/p: brush
print(numbers[3]) #o/p: 87
print(grocery[0],numbers[0]) #o/p: Soap 43

print('No. of items in Grocery list: ',len(grocery)) #7
print('No. of items in Numbers list: ',len(numbers)) #11
print(numbers.count(43)) #returns instances of 43 as 2.
grocery.append('Salt')
print(grocery)

'''Never pass fn like 'append', 'sort' etc. directly in 'print'; the new value gets appended in list but the list
is not printed ('None' gets printed). Ref next two lines.'''
print(numbers.append(101)) #prints 'None' but appends 101 anyway
print(numbers) #o/p: [43, 21, 43, 87, 56, 34, 6, 90, 65, 13, 28, 100, 101]

'''Following 2 lines are correct way to append to list and print it'''
numbers.append(100)
print(numbers)

numbers.sort()
print(numbers) #o/p: [6, 13, 21, 28, 34, 43, 43, 56, 65, 87, 90]
numbers.reverse()
print(numbers) #o/p: [90, 87, 65, 56, 43, 43, 34, 28, 21, 13, 6]

grocery.sort()
print(grocery) #o/p: ['Brush', 'Chair', 'Eraser', 'Fish', 'Kite', 'Pen', 'Soap']
grocery.reverse()
print(grocery) #o/p: ['Soap', 'Pen', 'Kite', 'Fish', 'Eraser', 'Chair', 'Brush']

print(numbers)
print(numbers[0:5]) #o/p: [43, 21, 43, 87, 56]
print(numbers[1:]) #o/p: [21, 43, 87, 56, 34, 6, 90, 65, 13, 28]
print(numbers[1:7]) #o/p: [21, 43, 87, 56, 34, 6]

'''In next 2 lines, 'numbers' gets sorted in asc and then printed in reverse.'''
numbers.sort()
print(numbers[::-1]) #o/p: [90, 87, 65, 56, 43, 43, 34, 28, 21, 13, 6]
print(numbers[:]) #o/p: [6, 13, 21, 28, 34, 43, 43, 56, 65, 87, 90]

print(numbers[1:5:2]) #o/p: [21, 87]

print(min(numbers)) #o/p: 6
print(min(grocery)) #o/p: Brush
print(max(numbers)) #o/p: 90
print(max(grocery)) #o/p: Soap

grocery.append(20) #adds 20 to end of 'grocery'
print(type(grocery[6])) #o/p: <class 'str'>
print(type(grocery[7])) #o/p: <class 'int'>
numbers.append('knife')
print(numbers)#o/p:[43, 21, 43, 87, 56, 34, 6, 90, 65, 13, 28, 'knife']

grocery.insert(1,'Zebra') #insert(index, NewItem)
print(grocery) #O/P: ['Soap', 'Zebra', 'Brush', 'Chair', 'Kite', 'Fish', 'Pen', 'Eraser']

numbers.remove(90) #removes 90 from 'numbers' list
print(numbers)

numbers.pop() #'pop' removes last added item from the list
print(numbers)
grocery.pop()
print(grocery)

num = numbers.copy()
print(num)
print(numbers) #same o/p as above
grocery.extend(numbers)

```

```

print(grocery)
'''OUTPUT: ['Soap', 'Brush', 'Chain', 'Kite', 'Fish', 'Pen', 'Eraser', 43, 21, 43, 87, 56, 34, 6, 90, 65, 13,
28]'''
grocery.clear()
print(grocery) #o/p: []

'''----Next, list concatenation-----'''
##list1=[1,2,3]
##list2=['A','B','C']
##print(list1+list2) #o/p:[1, 2, 3, 'A', 'B', 'C']
##print(list1+['a','b','c']) #o/p:[1, 2, 3, 'a', 'b', 'c']
##
'''Deleting from lists'''
##del list2[2]
##print(list2)
##list2.append('C')
##print(list2)

##list3=[1,2,3,4,5,6,7]
##print(list3)
##for i in range(len(list3)-1,0,-1):
##    del list3[i]
##    print(list3)
'''O/P:
[1, 2, 3, 4, 5, 6, 7]
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5]
[1, 2, 3, 4]
[1, 2, 3]
[1, 2]
[1]
'''

'''-----Storing a list of cat names input by user-----'''
##catNames=[]
##while True:
##    print('Enter name of cat '+str(len(catNames)+1)+' (or ENTER to stop).')
##    name=input()
##    if name=='': break
##    catNames=catNames+[name] #list concatenation
##print('The cat names are: ')
##for i in catNames: print(' '+i)

##for i in range(len(catNames)): print('Cat no. '+str(i+1)+' is: '+catNames[i])

'''-----Multiple Assignment Trick-----'''
##cat = ['Fat','Black','Loud']
##size=cat[0]
##color=cat[1]
##disposition=cat[2]
##print(size,color,disposition) #o/p:Fat Black Loud
#OR
##size,color,disposition = cat
##print(size,color,disposition) #same o/p as above 'print'

##eggs=[1,2,3]
##del eggs[2]
##del eggs[1]
##del eggs[0]
##eggs.append(4)
##eggs.append(5)
##eggs.append(6)
##print(eggs)

##spam=[0,1,2,3,4,5]
##cheese=spam #assigning ref to 'cheese' NOT copying spam to cheese i.e.cheese is same as spam
##print(cheese) #o/p:[0, 1, 2, 3, 4, 5]
##cheese[1]='Hey!'
##print(cheese) #o/p:[0, 'Hey!', 2, 3, 4, 5]
##print(spam) #o/p same as above

```

```

##spam=['A','B','C','D']
##cheese=spam.copy() #separate copy of spam created in cheese
##print(cheese) #o/p:['A', 'B', 'C', 'D']
##cheese[1]=100
##print(cheese) #o/p:['A', 100, 'C', 'D']
##print(spam) #o/p:['A', 'B', 'C', 'D']

'''Practice Project of Sweigart's Book on P-71'''
def listingItems(li):
    for i in range(0,len(li)):
        print(li[i]+',',end=' ')
        if i==len(li)-2: print('and '+li[i+1]); break
spam=['apples', 'bananas','tofu','cats']
print('The list is: ',spam) #o/p: ['apples', 'bananas','tofu','cats']
listingItems(spam) #o/p: apples, bananas, tofu, and cats

```

```

'''A generic version of the above program-----'''
def listingItems(li):
    for i in range(0,len(li)):
        print(li[i]+',',end=' ')
        if i==len(li)-2: print('and '+li[i+1]); break
spam=[]
while True:
    choice=input('Enter item to list. Enter \'q\' or \'Q\' to stop adding items.')
    if choice=='q' or choice=='Q': break
    else: spam+= [choice]
print('The list is: ',spam)
print('The items in list are:')
listingItems(spam)

```

```

'''String characters broken into list items-----'''
def strToList(stl):
    for i in range(0,len(stl)):
        print(stl[i], end=' ')
        if i==len(stl)-2: print(stl[i+1]); break

user_string=input('Enter a word: ')
print('The string in list format is: '+str(list(user_string)))
print('Items of the list are: ')
strToList(list(user_string))

```

-----TUPLES-----

```

#Tuples are immutable

##eggs=('hello',42,0.5)
##print(type(eggs)) #o/p:<class 'tuple'>
##eggs[1]=99
##print(eggs) #ERROR:'tuples' don't support item assignment

##print(type('hello')) #o/p:<class 'str'>
##print(type(('hello',))) #o/p:<class 'tuple'>

'''Converting data types with 'list()' & 'tuple()' fns'''
##tup1=(['cat','dog',5])
##print(tup1) #o/p:['cat', 'dog', 5]
##print(list(tup1)) #o/p:['cat', 'dog', 5]
##print(list('hello')) #o/p:['h', 'e', 'l', 'l', 'o']

```

-----DICTIONARIES-----

```

'''Dictionary is MUTABLE, just like Lists. Tuple is NOT mutable. Dictionary is key-value pairs'''
d1 = {}
##print(type(d1)) #o/p: <class 'dict'>
##print(len(d1)) #o/p: 0 ; coz 0 key-value pairs

```

```

d2 = {'K1':'Burger','K2':'Fish','K3':'Roti'}
##print(len(d2)) #o/p: 3 ; coz 3 key-value pairs
##print(d2.values()) #o/p:dict_values(['Burger', 'Fish', 'Roti'])
##print(d2.keys()) #o/p:dict_keys(['K1', 'K2', 'K3'])
##print(d2.items()) #o/p:dict_items([('K1', 'Burger'), ('K2', 'Fish'), ('K3', 'Roti')])
##print(d2['K2']) #o/p: Fish

##for key in d2: print(key) #o/p:K1,K2,K3
##for key in d2: print(d2[key]) #o/p: Burger, Fish, Roti
##for key in d2: print(key,':',d2[key]) #prints key-value pair of d2

##d2['K4'] = 'Paneer'
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti', 'K4': 'Paneer'}

##del d2['K3']
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish'}
##d2[25] = 'Schawarma'
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish', 25: 'Schawarma'}

d3 = {'K1':'Thai','K2':'Indian', 'K3':{'m1':'kimchi','m2':'momo'}} #dictionary within dictionary
##d3 = {}
##print(d3) '''if 'd3={}'' line above is uncommented, d3 will be updated to empty dictionary and {} gets
printed in this print stmt'''

##print(d3['K3']) #o/p:{'m1': 'kimchi', 'm2': 'momo'}
##print(d3['K3']['m2']) #o/p: momo

##d4 = d2.copy() '''d4 is new COPY of d2; original d2 is preserved'''
##print(d4) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti'}
##del d4['K3']
##print(d4) #o/p:{'K1': 'Burger', 'K2': 'Fish'}
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti'}

'''In next 5 lines, d4 gets REFERENCE of d2 (d2 & d4 are now same dictionary). Thus, deleting
key K3 from d4 is akin to deleting K3 from d2. That's why the print statements (after del K3)
output same results.'''
##d4 = d2
##print(d4) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti'}
##del d4['K3']
##print(d4) #o/p:{'K1': 'Burger', 'K2': 'Fish'}
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish'}

##print(d2.get('K3')) #o/p: roti
##d2.update({'K4':'Kheer'})
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti', 'K4': 'Kheer'}
##d2.pop('K4') '''Any key-value pair can be popped'''
##print(d2) #o/p:{'K1': 'Burger', 'K2': 'Fish', 'K3': 'Roti'}

##del d2['Fish'] #print in next line will throw error coz 'fish' is NOT the key
##print(d2)

d5={1:'Apple',2:'Ball',3:'Cat'}
##print(d5.__contains__(1)) #o/p: True
##print(d5.__delitem__())
##d5.clear() '''clear' takes NO args & only empties dict, NOT del it. That's why 'len' in next line returns 0.
##print(len(d5)) #o/p: 0

##d6 = dict(d5) #creates a copy d6 of d5
##print(d6)
##del d6[3] #deletes '3:Cat' in d6; doesn't alter d5
##print(d6)
##print(d5)

'''Next is dictionary within dictionary'''
family1 = {
    "child1" : {
        "name" : "Emil",
        "year" : 2004
    },

```

```

"child2" : {
    "name" : "Tobias",
    "year" : 2007
},
"child3" : {
    "name" : "Linus",
    "year" : 2011
}
}
##print(family1['child1']['name']) #o/p:Emil
##for child in family1: print(child, end=' ') #o/p: child1 child2 child3
##for child in family1: print(family1[child])

'''Next, nesting of dictionaries'''
child1={'Name':'Emil', 'Year':'2004'}
child2={'Name':'Tobias', 'Year':'2007'}
child3={'Name':'Linus', 'Year':'2011'}
family2 = {'child1':child1, 'child2':child2, 'child3':child3}
##print(family2.values())
##print(family2.keys()) #o/p:dict_keys(['child1', 'child2', 'child3'])
##print(family2['child1']['Name'], family2['child1']['Year']) #o/p:Emil 2004

##for x in child1.keys(): print(str(x), ' : ',str(child1[x]))
'''OUTPUT:
Name   :   Emil
Year   :   2004
'''

##print(family2['child1'].values()) #o/p:dict_values(['Emil', '2004'])
##print(family2['child2'].values()) #o/p:dict_values(['Tobias', '2007'])
##print(family2['child3'].values()) #o/p:dict_values(['Linus', '2011'])

'''equal to above 3 'prints' can be done by 'for' loop'''
##for i in family2: print(family2[i].values()) #same result as above 3 'prints'

##for key,value in family2.items(): print(key,value)
##for key in family2.keys(): print(family2[key])
##for val in family2.values(): print(val)

##for x in family2:
##    print(x,family2[x]['Name'], end=" ")
##    print(family2[x]['Year'])
'''OUTPUT:
child1 Emil 2004
child2 Tobias 2007
child3 Linus 2011
'''

'''-----Dictionaries V/S Lists-----'''
Spam = ['cats','dogs','moose']
Bacon = ['dogs','moose','cats']
print(spam==bacon) #o/p: False

eggs={1001:'Tom', 1002:'Dick', 1003:'Harry'}
ham={1003:'Harry', 1001:'Tom', 1002:'Dick'}
print(eggs == ham) #o/p:True
cheese={1003:'Harry', 1001:'Tom', 1002:'Dicks'}
print(eggs == cheese) #o/p: False

'''-----Birthday Dictionary-----'''
bdays={'Alice':'Apr 1', 'Bob':'Dec 12', 'Carol':'Mar 4'}
##while True:
##    print('Enter a name: (press SPACE to quit)')
##    name=input()
##    if name==' ': break
##    elif name in bdays: print(bdays[name]+' is birthday of '+name+'.')
##    else:
##        print(name+' has no record of birthday. What is their birthday?')
##        newdob=input()
##        bdays[name]=newdob

```

```

##         print('Birthdays DB updated...')

##print(bdays.items())
##for i in bdays.items(): print(i)
##for i in bdays.keys(): print(i)
##for i in bdays.values(): print(i)

'''-----Quickly revise Dictionaries-----'''
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
#print(thisdict)
#print(thisdict.get('brand'))
#thisdict["year"]=1965
#print(thisdict)
#for x in thisdict: print(x)
#for x in thisdict: print(thisdict[x])
#for x in thisdict.keys(): print(x)
#for x in thisdict.values(): print(x)
#for x in thisdict.keys(): print(str(x)+' : '+str(thisdict[x]))
#thisdict['color']='red'
#print(thisdict)

#thisdict.pop('year')
#print(thisdict)
#thisdict['year']='1965'
#print(thisdict)
#thisdict.popitem() #deletes the last inserted item from dictionary
#print(thisdict)

#del thisdict #deletes dictionary structure permanently
#print(thisdict) #will throw error as dictionary is absent

#thisdict.clear() #only deletes content of dictionary
#print(thisdict) #displays empty dictionary

#mydict = thisdict.copy()
#print(mydict)
#mydict=thisdict
#print(mydict) #same o/p as above

#x=('key1','key2','key3')
#y=0
#mydict=dict.fromkeys(x,y)
#print(mydict)

#x=('key1','key2','key3')
#y=(10,12)
#mydict=dict.fromkeys(x,y)
#print(mydict)

##spam={'color':'red', 'age':42}
##for k,v in spam.items():
##    print(k,v)
##    print('Key :'+str(k),', Value: '+str(v))
'''OUTPUT:
Key :color , Value: red
Key :age , Value: 42
'''

'''The 'get()' method has two args: the key of the value to retrieve & a fallback value to return if
key doesn't exist.'''
partyItems={'apples':5,'cups':2}
print('Bring '+str(partyItems.get('cups',0))+ ' cups to party.') #o/p: Bring 2 cups to party.
print('Bring '+str(partyItems.get('eggs',0))+ ' eggs to party.') #o/p: Bring 0 eggs to party.

'''Nested Dictionaries: Page-81, Sweigart'''
def totalBrought(guests,item):

```

```

numBrought=0
for k,v in guests.items(): numBrought = numBrought+v.get(item,0) #'get' fn fetches value of key 'item'
    (assigns value 0 in case key has no value)
return numBrought

allGuests={'Alice':{'Apples':5, 'Pretzels':12},'Bob':{'HamSandwiches':3, 'Apples':2},'Carol':{'Cups':3,
    'ApplePies':1}}
print('Number of things brought:')
print(' --Apples      ' + str(totalBrought(allGuests, 'Apples'))))
print(' --Cups        ' + str(totalBrought(allGuests, 'Cups'))))
print(' --Cakes        ' + str(totalBrought(allGuests, 'Cakes'))))
print(' --Ham s/w      ' + str(totalBrought(allGuests, 'HamSandwiches'))))
print(' --Apple Pies   ' + str(totalBrought(allGuests, 'ApplePies'))))

'''O/P:
Number of things brought:
--Apples      7
--Cups        3
--Cakes        0
--Ham s/w      3
--Apple Pies  1
'''

'''Inventory: Page-82 Sweigart'''
def displayInventory(mystuff):
    itemTotal=0
    print('----INVENTORY----')
    for k,v in mystuff.items(): print(str(v)+' '+k); itemTotal += v
    print('Total items: '+str(itemTotal))

stuff = {'rope':1, 'torch':6, 'gold coin':42, 'dagger':1, 'arrow':12}
displayInventory(stuff)
'''O/P:
----INVENTORY----
1 rope
6 torch
42 gold coin
1 dagger
12 arrow
Total items: 62
'''

'''Adding list items to dictionary (Very Imp.)'''
##def addToInventory(li,dic):
##    for i in li:
##        if i not in dic: dic[i]=1
##        else: dic[i]=dic[i]+1
##    print(dic)
##
##def displayInventory(inventory):
##    print('The inventory is: ')
##    for k,v in inventory.items(): print(k+' : '+str(v))
##    print('Total Items:'+str(sum(inventory.values())))
##
updict={'gold coin':42, 'rope':1}
lis=['rope','gold coin', 'dagger','ruby']
##addToInventory(lis,updict)
##displayInventory(updict)

##OUTPUT:
{'gold coin': 43, 'rope': 2, 'dagger': 1, 'ruby': 1}
The inventory is:
gold coin : 43
rope : 2
dagger : 1
ruby : 1
Total Items:47

```

-----REGULAR EXPRESSIONS-----

```
'''FUNCTIONS: compile, search (returns a Match obj if there is a match in string), group, groups,
findall (returns list containing all matches), split (returns a LIST where string has been
split at each match), sub (replaces one or many matches with a string)'''
```

```
'''The search() method returns object of the FIRST matched text in the searched string; if there is no match,
NONE is returned.'''
```

```
# import re
```

```
# txt = "The rain in Spain"
# x = re.search("Portugal", txt)
# print(x) #o/p: None
```

```
# txt = "The rain in Spain"
# x = re.search("ain", txt)
# print(x) #o/p: <re.Match object; span=(5, 8), match='ain'>
# print(x.group()) #o/p: ain ; group() returns the string obtained in 'match' attrib from above o/p
```

```
# txt = "The rain in Spain"
# x = re.search(r"\bS\w+", txt) #one or more words starting with 'S'
# print(x.group()) #o/p: Spain; #FIRST instance of matched pattern from string is returned
# print(x.string) #o/p: The rain in Spain; x.string returns the string ('txt' here) which contains match
```

```
import re #import module for regex
```

```
# pattern = re.compile(r'\d{10}') #Passing a string value representing your regular expression to re.compile()
# returns a pattern object (or simply, a Regex object).
# cellNumber = pattern.search('My number is: 8007725198.')
# print('Phone number found: '+cellNumber.group())
'''OUTPUT: Phone number found: 8007725198'''
```

```
# pattern = re.compile(r'\d\d\d\d\d\d\d\d\d\d')
# cellNumber = pattern.search('My number is: 8007725198.')
# print('Phone number found: '+cellNumber.group())
'''OUTPUT: Phone number found: 8007725198'''
```

```
'''The group() fn returns entire regex; group(i) returns ith group of regex; the groups() fn returns
TUPLES of groups of the regex. See the following example.'''
```

```
pattern2=re.compile(r'(\d\d\d)-(\d\d\d-\d\d\d\d)')
cellNo=pattern2.search('My no. is 415-555-4252')
# print(cellNo.group(1)) #o/p:415
# print(cellNo.group(2)) #o/p:555-4252
# print(cellNo.group(0)) #o/p:415-555-4252
# print(cellNo.group()) #o/p:415-555-4252
# print(cellNo.groups()) #o/p:('415', '555-4252')
# areaCode, mainNumber = cellNo.groups() #creates tuples of groups & assigns (in order) to variables on LHS
# print(areaCode) #o/p: 415
# print(mainNumber) #o/p: 555-4252
```

```
'''Including parentheses in regex'''
```

```
pattern3 = re.compile(r'((\d\d\d\d)-(\d\d\d\d-\d\d\d\d))')
cellNo3 = pattern3.search('The new number is: (415)-555-4252.')
# print('New number found is: '+cellNo3.group()) #o/p:New number found is: (415)-555-4252
# print('New no. found is :',cellNo3.groups()) #o/p:New no. found is : (('415)-555-4252', '555-4252')
```

```
'''Matching Multiple Groups With Pipe (|) character'''
```

```
'''In next example, pipe character will match EITHER 'Batman' or 'Tina Fey'. When both 'Batman' and 'Tina Fey'
occur together, the first occurrence of matching text will be returned.'''
```

```
# actors = re.compile(r'Batman|Tina Fey')
# actor = actors.search('Batman and Tina Fey')
# print('Actor: ',actor.group()) #o/p:Actor: Batman
# actor1 = actors.search('Tina Fey and Batman')
# print('Actor: ',actor1.group()) #o/p:Actor: Tina Fey
```

```

actors2 = re.compile(r'Bat(man|mobile|copter)')
actor2 = actors2.search('Batmobile lost a wheel.')
# print(actor2.group()) #o/p: Batmobile
# print(actor2.group(1)) #o/p: mobile
'''In above 2 print stmts, group() returns the full matched text 'Batmobile', while group(1) returns just the
part of the matched text inside the first parentheses group, 'mobile' '''

```

'''Matching exactly ZERO OR ONE instances with '?' character'''

```

# bat = re.compile(r'Bat(wo)?man') #the '?' enables to match 0 or 1 instances of 'wo'
# hero = bat.search('The adventures of Batman')
# print(hero.group()) #o/p: Batman
# hero1 = bat.search('The adventures of Batwoman')
# print(hero1.group()) #o/p: Batwoman
# hero2 = bat.search('No hero here.') #since no string here matches with 'bat', 'NoneType' is assigned to 'hero2'
# print(hero2.group()) #since 'hero2' has NoneType (above comment), group() doesn't work for it
# hero3 = bat.search('Batwomania')
# print(hero3.group()) #o/p: Batwoman
# hero4 = bat.search('Batimania')
# print(hero4.group()) #same error as for 'hero2' coz of wrong string entered in above stmt

# phoneNum = re.compile(r'(\d\d\d-)?\d{3}-\d{4}') #matching 0 or 1 group preceding question mark
# number = phoneNum.search('This number: 415-555-4242')
# print(number.group()) #o/p: 415-555-4242
# number1 = phoneNum.search('This number: 555-4242')
# print(number1.group()) #o/p: 555-4242

```

'''Matching ZERO OR MORE instances with '*' sign'''

```

# bat = re.compile(r'Bat(wo)*man') #the '?' enables to match 0 or more instances of 'wo'
# hero = bat.search('Batwowomania')
# print(hero.group()) #o/p: Batwowoman
# hero1 = bat.search('Batwoman')
# print(hero1.group()) #o/p: Batwoman

```

'''Matching ONE OR MORE instances with '+' sign'''

```

bat = re.compile(r'Bat(wo)+man') #the '?' enables to match 0 or more instances of 'wo'
# hero = bat.search('Batmania') #assigns NoneType to 'hero' as at least ONE 'wo' expected.
# print(hero.group()) #throws error due to above comment
hero1 = bat.search('Batwoman')
print(hero1.group()) #o/p: Batwoman

```

'''Matching specific repetitions with '{}''''

```

#E.g. the regex '(Ha){3}' will match 'HaHaHa' and not 'HaHa'
#Regex '(Ha){3,5}' will match 3,4, or 5 repetitions of 'Ha'
#Regex '(Ha){3,}' will match 3 or more repetitions
#Regex '(Ha){,5}' will match 0 to 5 repetitions
# ha = re.compile(r'(Ha){3}')
# matcher = ha.search('HaHaHaHa')
# print(matcher.group()) #o/p: HaHaHa

```

```

# ha1 = re.compile(r'(Ha){3,}') #greedy way
# matcher1 = ha1.search('HaHaHaHa')
# print(matcher1.group()) #o/p: HaHaHaHa; 3 or more repetitions of 'Ha' will always yield longest repetition
(greedy way)

```

```

# ha2 = re.compile(r'(Ha){,5}')
# matcher2 = ha2.search('HaHaHaHaHaHa')
# print(matcher2.group()) #o/p: HaHaHaHaHa; at most 5 repetitions are printed

```

```

# ha3 = re.compile(r'(Ha){3,5}??') #non-greedy way
# matcher3 = ha3.search('HaHaHaHa') #due to '?' added in above stmt, shorter repetition of 'Ha' is printed
# print(matcher3.group())
# ha4 = re.compile(r'(Ha){3,}??')
# matcher4 = ha4.search(' ') #even if something (other than 'Ha') was type, o/p in next line remains same
# print(matcher4.group()) #o/p: (nothing printed)

```

'''The findall() Method'''

```

#This method will return the strings of EVERY match in searched string.
# txt = "The rain in Spain"
# x = re.findall("ai", txt)

```

```
# print(x) #o/p:['ai', 'ai']
```

```
# txt = "The rain in Spain"
# x = re.findall("rain", txt)
# print(x) #o/p: ['rain']
```

Following example explains how search() is diff from findall()

```
# phoneNumRegex = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')
# match = phoneNumRegex.search('Cell: 415-555-4242 Work:212-555-0000')
# print(match.group()) #o/p: 415-555-4242 (this is 1st instance of matched string)
```

```
# cellNum = re.compile(r'\d\d\d-\d\d\d-\d\d\d\d')
# print(cellNum.findall('Cell: 415-555-4242 Work:212-555-0000')) #o/p: ['415-555-4242', '212-555-0000']
# findall() returns ALL instances (as a list of strings) that match the specified pattern in compile()
```

```
# cellNum2 = re.compile(r'(\d\d\d)-(\d\d\d)-(\d\d\d\d)')
# print(cellNum2.findall('Cell: 415-555-4242 Work:212-555-0000')) #o/p: [('415', '555', '4242'), ('212', '555',
'0000')]
```

#when the regex is grouped within parentheses in compile(), findall() returns list of tuples

'''The split() Function'''

The split() function returns a list where the string has been split at each match:

```
# txt = "The rain in Spain"
# x = re.split("\s", txt)
# print(x) #o/p: ['The', 'rain', 'in', 'Spain']
```

```
# txt = "The rain in Spain"
# x = re.split("\s", txt, 1) #split at 1st space in 'txt'
# print(x) #o/p: ['The', 'rain in Spain']
```

'''Substituting strings with sub() method'''

```
# txt = "The rain in Spain"
# print(re.sub('s','9',txt)) #o/p: The9rain9in9Spain
# print(re.sub('s','9', txt, 1)) #o/p: The9rain in Spain; observe how ONLY 1st space is replaced by 9
```

```
# pattern = re.compile(r'Agent \w+')
# name = pattern.sub('CENSORED', 'Agent Alice betrayed Agent Bob.')
# print(name) #o/p:CENSORED betrayed CENSORED.
```

```
# pattern = re.compile(r'Agent (\w)\w*')
# agent = pattern.sub(r'\1***', 'Agent Carol killed Agent Bob.') #match above pattern & substitute, while keeping
first letter of word after 'Agent' unchanged
# print(agent) #o/p: C*** killed B***.
```

'''Complex regex patterns can be written in simple form by telling compile() to ignore whitespace & comments inside pattern. This 'verbose mode' can be enabled by passing the variable re.VERBOSE as second arg to re.compile()'''

```
#phonePattern=re.compile(r'''(
#(\d{3} | \(\d{3}\))? (\s|-|\.)?
#\d{3}
#(\s|-|\.)
#\d{4}
#(\s*(ext|x|ext.) \s*\d{2,5})?
#)''',re.VERBOSE)
#phoneStmt = 'My number is: 420-555-3214 ext. 12547'
#results = phonePattern.finditer(phoneStmt)
#for result in results: print(result) #o/p: <re.Match object; span=(14, 37), match='420-555-3214 ext. 12547'>
```

'''The span() function'''

#It returns TUPLE having start & end pos of match in string'''

```
# txt = "The rain in Spain"
# x = re.search(r"\bS\w+", txt)
# print(x.span()) #o/p:(12, 17)
```

'''Character Classes'''

*#\d: numeric (0 to 9); \D: any char not numeric digit; \w: any letter, numeric digit or underscore;
\W: any char NOT a letter, digit or underscore; \s: space, tab or newline; \S: any char NOT space, tab or
newline*

```
# xmas = re.compile(r'\d+\s\w+')
# print(xmas.findall(' the 3 baubles & 1 tree')) #o/p:['3 baubles', '1 tree']
```

'''Making your own character classes'''

#Define your own char class using square brackets; e.g. [aeiouAEIOU] will match any vowel.

'''In square brackets, normal reg exprsn symbols are not interpreted as such i.e. you don't need to escape a char with backslash'''

```
# vowel = re.compile(r'[aeiouAEIOU]')
# print(vowel.findall('Aeronautical Engineer')) #o/p:['A', 'e', 'o', 'a', 'u', 'i', 'a', 'E', 'i', 'e', 'e']
```

```
# sentence = re.compile(r'[a-zA-Z0-9]') #hyphen used to specify range of all lowercase, uppercase & numbers
# print('Appledore towers.') #o/p: Appledore towers
```

'''Placing caret (^) just after character class opening bracket creates negative character class which matches all characters NOT in the character class'''

```
consonant = re.compile(r'^[aeiouAEIOU]')
print(consonant.findall('Lazy Dog')) #o/p:['L', 'z', 'y', ' ', 'D', 'g']
```

'''Caret (^) & Dollar (\$) to search the string at start or end. Caret always comes before dollar.'''

```
# beginsWith = re.compile(r'^Hello') #string starting with Hello
# print(beginsWith.search('Hello World')) #o/p:<re.Match object; span=(0, 5), match='Hello'>
# endsWith = re.compile(r'\d.$') #string ending in a numeric digit followed by period
# print(endsWith.search('He is 46.')) #o/p:<re.Match object; span=(7, 9), match='6.'>
# numString = re.compile(r'^\d+$') #string starting with & ending in at least ONE num digit
# print(numString.search('123456')) #o/p:<re.Match object; span=(0, 6), match='123456'>
```

'''The Dot (.) character (WILDCARD) matches any ONE character except a newline'''

```
# atRegex = re.compile(r'.at') #any string ending in 'at'
# print(atRegex.findall('The cat in the hat sat on flat mat.')) #o/p:['cat', 'hat', 'sat', 'lat', 'mat']
# print(atRegex.findall('Flat swat cat')) #o/p:['lat', 'wat', 'cat']
```

'''Match anything with Dot-Star'''

'''The dot-star uses greedy mode by always trying to match as much text as possible. Dot means "any single char except newline" and star char means "zero or more of the preceding character"'''

```
# name = re.compile(r'FirstName:(.*) LastName:(.*)')
# myname = name.search('FirstName: Al LastName: Sweigart')
# print(myname.group(1)) #o/p: Al
# print(myname.group(2)) #o/p: Sweigart
'''Use '.*?' to match any and all text in non-greedy fashion'''
```

'''In next 2 examples, both regexes roughly translate to "Match an opening angle bracket, followed by anything, followed by a closing angle bracket." But the string '<To serve man> for dinner.>' has two possible matches for the closing angle bracket. In the nongreedy version of regex, Python matches shortest possible string: '<To serve man>'. In the greedy version, Python matches the longest possible string: '<To serve man> for dinner.>'.'''

```
# nongreedy = re.compile(r'<.*?>')
# mo = nongreedy.search('<To serve man> for dinner.>')
# print(mo.group()) #o/p: <To serve man>
```

```
# greedy = re.compile(r'<.*>')
# md = greedy.search('<To serve man> for dinner.>')
# print(md.group()) #o/p: <To serve man> for dinner.>
```

'''Matching newlines with the dot character (by passing 're.DOTALL' as 2nd arg to compile())'''

```
# noNewLine = re.compile('.*') #will search only till 1st newline char is found
# noLine = noNewLine.search('Apple \n Doctor')
# print(noLine.group()) #o/p: Apple
```

newline = re.compile('.*', re.DOTALL) #will match EVERYTHING

```
# new = newLine.search('Apple.\nDoctor\n')
# newLine.search('Apple.\nDoctor\n').group() #run this line ONLY in console; o/p is 'Apple.\nDoctor\n' which has
# newline char as well
```

```
# print(new.group())
```

```
'''o/p:
Apple.
Doctor'''
```

'''-----Case insensitive matching using IGNORECASE or I-----'''

```
# Robocop = re.compile(r'roBocOp', re.IGNORECASE)
# print(robocop.search('RoBoCoP is part man, part machine, all cop.').group()) #o/p: RoBoCoP
```

SHORT EXAMPLES TO CLARIFY CONCEPTS:

```
>>> import re
>>> pattern = re.compile(r'\w') #match any word char
>>> string = 'Start a sentence & end it.'
>>> pattern.search(string) #search() returns NON-GREEDY result; the first instance of a match
<re.Match object; span = (0, 1), match='S'>
>>> pattern.findall(string) #findall() returns GREEDY result; all the matches
['S', 't', 'a', 'r', 't', 'a', 's', 'e', 'n', 't', 'e', 'n', 'c', 'e', 'e', 'n', 'd', 'i', 't']

>>> pattern = re.compile(r'\w+') #match one or more words
>>> pattern.search(string) #returns FIRST instance of match (here, the first word)
<re.Match object; span=(0, 5), match='Start'>
>>> pattern.findall(string) #returns ALL matches
['Start', 'a', 'sentence', 'end', 'it'] #observe how '&' in 'string' is taken as special char & not printed

>>> pattern = re.compile(r'\w*') #match zero or more words
>>> pattern.search(string) <re.Match object; span = (0, 5), match='Start'>
>>> pattern.findall(string) ['Start', '', 'a', '', 'sentence', '', '', '', 'end', '', 'it', '', '']

>>> pattern = re.compile(r'\W') #NOT a word char
>>> string = 'Start a sentence and end it.'
>>> pattern.search(string) <re.Match object; span=(5, 6), match=' '> #returns FIRST match that is NOT a word char
>>> pattern.findall(string) #returns ALL matches that are NOT word chars
[' ', ' ', ' ', ' ', ' ', ' ', ' ', '.'] #note the full-stop included in the o/p.

>>> string = 'Ha hahaha Ha HaHa ha'
>>> pattern = re.compile(r'\bha') #the 'ha' with space (boundary) b4 it.
>>> match = pattern.finditer(string) #find all instances of 'ha' with space (boundary) b4 it
>>> for matches in match: print(matches) #iterate through all the found instances
<re.Match object; span=(3, 5), match='ha'>
<re.Match object; span=(18, 20), match='ha'>
>>> pattern = re.compile(r'ha\b') #the 'ha' with a space (boundary) after it
>>> match = pattern.finditer(string)
>>> for matches in match: print(matches)
<re.Match object; span=(7, 9), match='ha'>
<re.Match object; span=(18, 20), match='ha'>

>>> pattern = re.compile(r'\Bha') #the 'ha' WITHOUT space (boundary) before it
>>> match = pattern.finditer(string)
>>> for matches in match: print(matches)
<re.Match object; span=(5, 7), match='ha'>
<re.Match object; span=(7, 9), match='ha'>

>>> pattern = re.compile(r'ha\b') #the 'ha' WITHOUT a space (boundary) after it
>>> match = pattern.finditer(string)
>>> for matches in match: print(matches)
<re.Match object; span=(3, 5), match='ha'>
<re.Match object; span=(5, 7), match='ha'>

>>> string = 'This is an apple.'
>>> pattern = re.compile(r'^This') #find if 'The' is at START (^) of string
>>> match = pattern.finditer(string)
>>> for matches in match: print(matches) <re.Match object; span=(0, 4), match='This'>
>>> pattern = re.compile(r'apple.$') #find if 'apple.' is at END ($) of string
>>> match = pattern.finditer(string)
>>> for matches in match: print(matches) <re.Match object; span=(11, 17), match='apple.'>

>>> cellno = '321-555-4321 123.555.1234'
>>> pattern = re.compile(r'\d{3}.\d{3}.\d{4}') #the dot character matches any char except newline; result would be same
# if we wrote the pattern as (r'\d{3}[-.]d{3}[-.]d{4}')
>>> match = pattern.finditer(cellno)
>>> for matches in match: print(matches)
<re.Match object; span=(0, 12), match='321-555-4321'>
<re.Match object; span=(13, 25), match='123.555.1234'>
>>> pattern.search(cellno) <re.Match object; span=(0, 12), match='321-555-4321'> #returns the FIRST match
>>> pattern.findall(cellno) ['321-555-4321', '123.555.1234']

>>> cellno = '800-123-1234 900*321*1234'
>>> pattern = re.compile(r'[89]00[-*]\d{3}[-*]1234') #begin wid 800 OR 900, then a dash/star...ending in 1234
>>> matches = pattern.finditer(cellno)
```

```

>>> for match in matches: print(match)
<re.Match object; span=(0, 12), match='800-123-1234'>
<re.Match object; span=(13, 25), match='900*321*1234'>

>>> num='91782564682'
>>> pattern = re.compile(r'[1-5]') #match digits 1,2,3,4 & 5 in num
>>> matches = pattern.finditer(num)
>>> for match in matches: print(match)
<re.Match object; span=(1, 2), match='1'>
<re.Match object; span=(4, 5), match='2'>
<re.Match object; span=(5, 6), match='5'>
<re.Match object; span=(7, 8), match='4'>
<re.Match object; span=(10, 11), match='2'>
>>> pattern.findall(num) ['1', '2', '5', '4', '2']
>>> pattern.search(num) <re.Match object; span=(1, 2), match='1'>

>>> string = 'zaq1ZA_{}Q!'
>>> pattern = re.compile(r'^[a-zA-Z]') #match chars which ARE NOT (^) upper or lower case in string
>>> matches = pattern.finditer(string)
>>> for match in matches: print(match)
<re.Match object; span=(3, 4), match='1'>
<re.Match object; span=(6, 7), match='_ '>
<re.Match object; span=(7, 8), match='{ '>
<re.Match object; span=(8, 9), match='}'>
<re.Match object; span=(10, 11), match='!'>

>>> names = 'Mr Tom, Mr. Jones, Mrs. Smith'
>>> pattern = re.compile(r'Mr[s]?[a-zA-Z]+' ) #same o/p for 'M(r|rs)\.[a-zA-Z]*' or 'M(r|rs)\.[a-zA-Z]+'
>>> matches = pattern.finditer(names)
>>> for match in matches: print(match)
<re.Match object; span=(0, 6), match='Mr Tom'>
<re.Match object; span=(8, 17), match='Mr. Jones'>
<re.Match object; span=(19, 29), match='Mrs. Smith'>

>>> emails = 'lock.Ho@gmail.com, tom-rid@dark-arts.net, ed@red.edu'
>>> pattern = re.compile(r'[a-zA-Z-]+@[a-z-]+\.(com|net|edu)') #same o/p for '[a-zA-Z-]+@[a-zA-Z-]+\.(com|net|edu)')
>>> matches = pattern.finditer(emails)
>>> for match in matches: print(match)
<re.Match object; span=(0, 17), match='lock.Ho@gmail.com'>
<re.Match object; span=(19, 40), match='tom-rid@dark-arts.net'>
<re.Match object; span=(42, 52), match='ed@red.edu'>

>>> urls = 'http://www.google.com, https://nasa.gov'
>>> pattern = re.compile(r'https?://(www\.)?[a-z]+\.[a-z]+') #'www.' is optional in 'urls'; 'https?://(www\.)?[a-z]+\.[a-z]+'
will give the same output
>>> matches = pattern.finditer(urls)
>>> for match in matches: print(match)
<re.Match object; span=(0, 21), match='http://www.google.com'>
<re.Match object; span=(23, 39), match='https://nasa.gov'>

>>> pattern = re.compile(r'https?://(www\.)?(\w+)(\.\w+)') #making groups
>>> matches = pattern.finditer(urls)
>>> for match in matches: print(match.group(0)) #group(0) returns groups as specified in pattern above
http://www.google.com
https://nasa.gov
>>> matches = pattern.finditer(urls)
>>> for match in matches: print(match.group(1))
www.
None
>>> matches = pattern.finditer(urls)
>>> for match in matches: print(match.group(2))
google
nasa
>>> subbed_urls = pattern.sub(r'\2\3', urls) #replace urls with only groups 2 and 3
>>> print(subbed_urls) #google.com, nasa.gov

>>> urls = 'http://www.google.com, https://nasa.gov'
>>> pattern = re.compile(r'https?://(www\.)?(\w+)(\.\w+)')
>>> matches = pattern.findall(urls) #the findall() method
>>> for match in matches: print(match)

```

```
('www.', 'google', '.com')
('', 'nasa', '.gov')
```

```
>>> urls = 'http://www.google.com, https://nasa.gov'
>>> pattern = re.compile(r'google')
>>> matches = pattern.match(urls) #match() matches the pattern at the START of string ('urls' here)
>>> print(matches) None #since there is no 'google' at the start of the string in variable 'urls'
>>> matches = pattern.search(urls) #search() matches through the ENTIRE string
>>> print(matches) <re.Match object; span=(11, 17), match='google'>
```

“STRONG PASSWORD DETECTION: 1 U’CASE, 1 L’CASE, 1 DIGIT & MIN LENGTH SHOULD BE 8”

```
import re
def pw_detection(pw):
    if len(pw) < 8: return False
    elif re.search('[0-9]', pw) is None: return False
    elif re.search('[A-Z]',pw) is None: return False
    elif re.search('[a-z]',pw) is None: return False
    else: return True
if __name__ == '__main__':
    pw = input('Enter your password: ')
    if password_detection(pw): print('Good, your password is strong!')
    else: print ('The password is at least eight characters long, contains both uppercase and lowercase
characters, and has at least one digit.')
```

REVIEW OF REGULAR EXPRESSIONS:

```
. : matches any char, except newline chars.
? : matches 0 or 1 of preceding pattern
* : matches 0 or more of preceding pattern
+ : matches 1 or more of preceding pattern
{n}: matches EXACTLY 'n' chars of preceding pattern
{n,}: matches n or more chars of preceding pattern
{,m}: matches 0 to m chars of preceding pattern
{n,m}: matches at least n and at most m chars of preceding pattern
{n,m}? or *? or +?: performs non-greedy match of preceding group
^spam: means string must begin with 'spam'
spam$: means string must end with 'spam'
| : either or (e.g 'falls|stays')
\d, \w, \s match a digit, word or space char respectively.
\D,\W,\S match anything EXCEPT a digit, word or space char respectively.
\A: returns a match if specified chars are at the beginning of string.
\b: returns a match where specified chars are at start or end of a word (word boundary i.e. space is present
before the match (if \b is used before the match))
\B: returns a match where specified chars are present, but NOT at start or end of a word
\s: returns a match where string has whitespace char
\S: returns a match where string DOES NOT has whitespace char
\w: returns a match when string contains any word char (from a-z, 0-9 & underscore)
\W: returns a match when string DOES NOT contain any word char
\Z: returns a match when specified chars are at end of string
[abc]: matches any char between brackets (a or b or c).
[^abc]: matches any char that ISN'T between brackets
[a-n]: returns a match for any lowercase char alphabetically between a and n
[0123]: returns a match if any of the digits (0,1,2 or 3) are present
[0-9]: returns a match for any digit between 0 & 9
[0-5][0-9]: matches any 2-digit num from 00 to 59
[a-zA-Z]: returns a match for any char alphabetically b/w a and z, lowercase or uppercase
```

-----FILE HANDLING-----

'''FUNCTIONS:

```
os.path.join(), os.getcwd(), os.chdir(), os.makedirs(), os.mkdir(), os.rename(), os.rmdir(), os.removedirs(),
os.path.abspath(), os.path.relpath(), os.path.isabs(), os.path.dirname(), os.path.basename(), os.path.split(),
os.path.splitext(),
os.stat(), os.walk().
```

FILE HANDLING FUNCTIONS: open(), read(), write(), close(), readlines()'''

```
import os
```

'''----- FILE PATHS -----'''

```
print(os.path.join('Hogwarts','Slytherin','Draco')) #o/p: Hogwarts\Slytherin\Draco
```

```
myfiles=['invite.docx', 'accounts.txt', 'details.csv']
```

```
for filename in myfiles: print(os.path.join('E:\Scripts\Python',filename))
```

'''OUTPUT:

E:\Scripts\Python\invite.docx

E:\Scripts\Python\accounts.txt

E:\Scripts\Python\details.csv

'''

#-----Current Working Directory (CWD)-----

'''There are 2 ways to specify file path. The ABSOLUTE PATH (which always begins with the root folder) and the RELATIVE PATH (which is relative to the program's CWD). The single dot \. for a folder name means 'this directory' whereas two dots \.. mean 'the parent folder'. Any filename or paths that don't begin with the root folder are assumed to be under CWD. You can get CWD as string value with os.getcwd() fn and change it with os.chdir(), however, error 'FileNotFound' will be thrown while changing a directory that doesn't exist'''

```
import os
```

```
print(os.getcwd()) #o/p: E:\Scripts\Python
```

```
os.chdir('E:\Scripts\Python\Files') #changes CWD to 'Files'
```

```
print(os.getcwd()) #o/p: E:\Scripts\Python\Files
```

```
os.chdir('E:\Scripts\Python') #changes CWD back to 'Python'
```

'''The 'os.makedirs()' function in next line creates all folders in path if they don't exist. Uncomment and run ONLY the 1st line (of the next 3 lines) to see the results in E drive. Then, comment the 1st line again. Next, uncomment & run the 2nd line and then comment it again. Then uncomment and run the 3rd line.'''

```
os.makedirs('E:\Test\Test1\Test2')
```

```
os.chdir('E:\Test\Test1\Test2') #this line has no effect on next line
```

```
os.removedirs('E:\Test\Test1\Test2') #os.rmdir() removes one file at a time (file at the end of path)
```

'''The above line will delete ALL the files in the path if they exist.

The below line will remove only 'Test2' file'''

```
os.rmdir('E:\Test\Test1\Test2')
```

'''Handling Absolute & Relative Paths Using abspath(), relpath() and isabs()'''

```
print(os.path.abspath('.')) #E:\Scripts\Python
```

```
print(os.path.abspath('.\Files')) #E:\Scripts\Python\Files
```

```
print(os.path.abspath('\Files')) #E:\Files (observe the . before '\Files' in above print stmt)
```

```
print(os.path.isabs('.\Files')) #o/p: True
```

```
print(os.path.isabs('\Files')) #o/p: False
```

```
print(os.path.isabs(os.path.abspath('.\Files'))) #o/p: True
```

```
print(os.path.isabs(os.path.abspath('.'))) #o/p: True
```

```
os.chdir('E:\Pensieve\SeverusDrive\movies\movv')
```

```
print(os.path.relpath('movv')) #o/p: '...\..\movv'
```

'''----- CREATING, REMOVING & TRAVERSING FILES -----'''

'''In next 3 lines, 'Test' folder is created, then renamed to 'Demo', & then finally deleted.'''

```
os.mkdir('E:\Test')
```

```
os.rename('E:\Test', 'E:\Demo')
```

```
os.rmdir('E:\Demo')
```

```
print(os.stat('E:\Test')) #create a 'Test' file and then only run this stmt
```

'''The os.stat() gives stats about directory'''

```
from datetime import datetime
```

```
file_modified_time=os.stat('E:\Test').st_mtime #attrib 'st_mtime' gives file's last modified time
```

```
print(datetime.fromtimestamp(file_modified_time)) #o/p:2020-10-15 20:27:05.833162
```

```

'''The os.walk() traverses directories and their contents. It returns 3 params: dir's path, dir's name and name of
files in dir'''
for dirpath, dirname, filename in os.walk('E:\Scripts\Python'):
    print('Current Path: ', dirpath)
    print('Directories: ', dirname)
    print('Files: ', filename)
    print('-----')

##print(os.startfile('E:\Geeta.pdf')) #opens file or folder in new window

'''FILE SIZES & FOLDER CONTENTS; Note: 'listdir()' is in 'os' module and NOT in 'os.path'.'''
print('Size in bytes: ', os.path.getsize('E:\Geeta.pdf'))
print(os.listdir('E:\Pensieve')) #o/p is a list of all folders/files inside 'Pensieve' folder

##totalSize = 0
##for filename in os.listdir('E:\Scripts\Python'):
##    totalSize = totalSize+os.path.getsize(os.path.join('E:\Scripts\Python', filename))
##print('Total size of files in Python dir is: ', totalSize) #o/p is total size in bytes of all files in Python
folder

'''-----CHECKING PATH VALIDITY-----'''
print(os.path.basename('\Script\Python')) #o/p: Python
print(os.path.dirname('\Script\Python')) #o/p: \Script
print(os.path.split('\Script\Python')) #o/p: ('\Script', 'Python') ; here dirname and basename are separated
print(os.path.exists('\Test\Test1')) #o/p: False ; (coz the specified path doesn't exist)
print(os.path.exists('..\Python')) #o/p: True ; (coz the path exists to 'Python' folder)
print(os.path.isdir('Python')) #o/p: False
print(os.path.isdir('..\Python')) #o/p: True
print(os.path.isfile('Python')) #o/p: False
print(os.path.isfile('..\Python')) #o/p: False
print(os.path.splitext('\demo\test.txt')) #o/p: ('\demo\test', '.txt') ; here file extensn is split from filepath
print('E:\Scripts\movy'.split(os.path.sep)) #o/p: ['E:', 'Scripts', 'movy']
'''In above line, string method 'split' is used along with 'os.path.sep' to return list of strings of folders in
path (drive E: is also a folder)'''

'''Determining if a DVD or a flash drive is currently attached to computer'''
print(os.path.exists('D://')) #o/p: False
print(os.path.exists('F://')) #o/p: True

```

```

'''-----FILE READING AND WRITING-----'''
There are 3 steps to reading or writing files in Python. Call open() to return 'File' object. Then call
read()/write() on that object. Finally, call close() to close that object. The file can have modes like
w,r,a & x modes.
'open(filepath, 'w')' opens the file for writing mode & creates the file if it is non-existent.
'open(filepath, 'r')' opens the file in reading mode which is DEFAULT mode (throws error if file is non-existential).
'open(filepath, 'a')' opens file in append mode & creates the file if it is absent.
'open(filepath, 'x')' simply creates a file and throws error if the file already exists.
Additionally, you can mention modes 'b' (for binary files like images) or 't' (text mode; this is the default
file extension).

```

Either use double (\\) for path or insert 'r' at start of path for the path to be validly accepted with only single (\) in it, else it throws error.

You can use (./) in path if you do not wish to do either of things mentioned in above comment

For example, the following line throws error:

```
newFile=open('E:\Scripts\Python\FileOps\nye.txt', 'x')
```

```
'''OSError: [Errno 22] Invalid argument: 'E:\\Scripts\\Python\\FileOps\\nye.txt' --> coz path has \n in '\nye.txt'
which is file name'''
```

NOTE: Here onward, uncomment & run the blocks of code in the order written and NOT randomly.

---- **Creating Files by Writing Path Correctly** ----

```

##f_hello = open('E:\\Scripts\\Python\\FileOps\\Hello.txt', 'x')
##f_bye = open(r'E:\Scripts\Python\FileOps\Bye.txt', 'x') #observe 'r' is inserted at path's start
##f_spam = open('E:/Scripts/Python/FileOps/Spam.txt', 'x')
##f_demo = open('./FileOps/Demo.txt', 'x')

```

----- **Writing To & Then Reading From Files** -----

Comment the above block of code. Now that files have been created in the above block, you can start reading from them. You can read entire contents of a file as a string value using the **read()** method.

As an example, consider the 'Hello.txt' file here.

```
f_hello = open('./FileOps/Hello.txt', 'w') #open file in 'w' mode; specifying mode 'a' will allow to append at the end of
the file ('w' mode overwrites everything)
f_hello.write('Hello World.\nGood to know you...') #write into the file
f_hello.close() #close the file
f_hello = open('./FileOps/Hello.txt', 'r') #open file in READ mode
read_hello = f_hello.read() #read file content into an object 'read_hello'
print(read_hello) #print the read content of file on console
f_hello.close() #you can also write this line b4 above line, but it's not a good practice.
```

Now, comment above block & run following block to APPEND to the file & print its content on console

```
f_hello = open('./FileOps/Hello.txt', 'a') #APPEND mode designated by 'a'
f_hello.write('\nHi! Just appending new stuff...\nFile handling ops\n are cool to know.\nKeep learning...')
#in above line, write() writes the string to file and RETURNS (if working directly on console) the no. of chars written including
newline.
f_hello.close() #close the file
f_hello = open('./FileOps/Hello.txt', 'r') #open file in READ mode
read_hello = f_hello.read() #read file content into an object 'read_hello'
print(read_hello) #print the read content of file on console
f_hello.close() #you can also write this line before prev line, and not here (but it is bad practice)
```

Now, comment the above block and run following block

```
f_hello = open('./FileOps/Hello.txt')
print(f_hello.readlines()) #returns a LIST of string values from file, one string for each line of text
f_hello.close()
'''output: ['Hello World.\n', 'Good to know you...\n', 'Hi! Just appending new stuff...\n', 'File handling ops\n', ' are cool to
know.\n', 'Keep learning...']
...
----- Deleting Files -----
```

```
os.remove('../FileOps/Hello.txt') #one can specify full absolute path also
```

----- Saving Variables with the 'shelve' Module -----

You can save variables in your Python programs to binary shelf files using the 'shelve' module. This helps the program to restore data to variables from the hard drive. This module lets you add 'Save' and 'Open' features to your program.

To read or write data using 'shelve' module, your first import 'shelve'. Then, call 'shelve.open()' and pass it a filename, and then store the returned shelf value in a variable. You can make changes to the shelf value as if it were a dictionary. When done, call 'close()' on the shelf value.

In the example below, our shelf value is stored in var 'shelve_file'. A list 'cats' is created. Then, we write 'shelve_file['cats']=cats' to store the list in var 'shelve_file' as a value associated with the key 'cats' (like a dictionary). Then we call close() on the 'shelve_file' variable.

```
import shelve
shelve_file = shelve.open('./FileOps/mydata') #IMP: shelf values don't need to be opened in r/w mode; can do both
once opened
cats = ['Zophie', 'Pooka', 'Simon']
shelve_file['cats'] = cats
shelve_file.close()
```

After running the previous code, comment it; you'll see three new files in the CWD (although we have provided another directory path above) which will be 'mydata.bak', 'mydata.dat' & 'mydata.dir'. These binary files contain the data you stored in your shelf.

In subsequent code, we check that our data was stored correctly. Just like dictionaries, shelf values have 'keys()' and 'values()' methods that will return list-like values instead of true lists. (If running on console, you must first pass them to the list() fn to get them in the list form.)

```
shelve_file = shelve.open('./FileOps/mydata')
print(type(shelve_file)) #o/p: <class 'shelve.DbfilenameShelf'>
print(shelve_file['cats']) #o/p: ['Zophie', 'Pooka', 'Simon']
shelve_file.close()
```

'Quiz Files' project of Albert Sweigart's book Automate The Boring Stuff With Python P-120

We are creating 5 quiz files and their corresponding answer key files as well. Each file has the same 5 MCQ type questions (questions are randomized in each quiz file; there are 4 options to every question, also randomized).

```
import os, random
#Create dictionary of states & their capitals
capitals = {'Punjab' : 'Chandigarh', 'Uttarakhand' : 'Dehradun', 'Bihar' : 'Patna', 'Rajasthan' : 'Jaipur',
            'Maharashtra' : 'Mumbai'}
for quizNum in range(5):
    #Creating the quiz and answer key files
    quizFile = open(r'E:\Scripts\Python\FileOps\QuizProject\capitalsquiz%s.txt' %(quizNum+1), 'w')
    answerKeyFile = open(r'E:\Scripts\Python\FileOps\QuizProject\capitalsquiz_answers%s.txt' %(quizNum+1), 'w')

    #Write out the header for the Quiz
    quizFile.write('Name:\n\nDate:\n\nPeriod:\n\n')
    quizFile.write((' '*20) + 'State Capitals Quiz (Form %s)' %(quizNum+1))
    quizFile.write('\n\n')

    #Shuffling the order of the states
    states = list(capitals.keys())
    random.shuffle(states) #shuffles the 'states' list's keys as mentioned in above line

    #Loop through all states, making a question for each
    for questionNum in range(5):
        #Get right & wrong answers.
        correctAnswer = capitals[states[questionNum]]
        wrongAnswers = list(capitals.values()) #creating a list containing all capitals
        del wrongAnswers[wrongAnswers.index(correctAnswer)] #correct answer removed from list 'wrongAnswers'
        wrongAnswers = random.sample(wrongAnswers, 3) #sample 3 random values from 'wrongAnswers'
        answerOptions = wrongAnswers+[correctAnswer] #creating list of 3 wrong & 1 right answers
        random.shuffle(answerOptions) #randomizing the 'answerOptions' list

        #Write the question and answer options to the quiz file.
        quizFile.write('%s. What is the capital of %s?\n' %(questionNum+1, states[questionNum]))
        for i in range(4):
            quizFile.write(' %s. %s\n' %('ABCD'[i], answerOptions[i])) #write A to D for i=0,1,2,3
        quizFile.write('\n')

        #Write the answer key to a file.
        answerKeyFile.write('%s. %s\n' %(questionNum+1, 'ABCD'[answerOptions.index(correctAnswer)]))
    quizFile.close()
    answerKeyFile.close()
```

#You can remove the files later on as follows:

```
## os.remove('./FileOps/QuizProject/capitalsquiz%s.txt' %(quizNum+1))
## os.remove('./FileOps/QuizProject/capitalsquiz_answers%s.txt' %(quizNum+1))
```

'''This is the Multiclipboard Project of Sweigart's chapter 8'''

```
import shelve, pyperclip, sys
mcbShelf = shelve.open('mcb')
```

#Saving the clipboard content

```
if len(sys.argv) == 3 and sys.argv[1].lower() == 'save': mcbShelf[sys.argv[2]] = pyperclip.paste()
elif len(sys.argv) == 2:
```

#List keywords and load content

```
    if sys.argv[1].lower() == 'list': pyperclip.copy(str(list(mcbShelf.keys())))
    elif sys.argv[1] in mcbShelf: pyperclip.copy(mcbShelf[sys.argv[1]])
mcbShelf.close()
```

'''-----RUNNING THE ABOVE PROGRAM-----'''

Type the following in the python SHELL:

```
import sys, os, pyperclip
```

#Below line readies the above program with 2 args 'save' and 'key1'.

```
sys.argv = ['E:/Scripts/Python/multiClipboard.pyw', 'save', 'key1']
exec(open('E:/Scripts/Python/Projects/Multiclipboard/multiClipboard.pyw').read())
```

#Previous line runs the above program with 2 args saving the clipboard content (if you have copied something) in key 'key1'.
 After the above line runs, current content of the clipboard is saved in 3 'mcb' files generated by above program.
 Next, you can Ctrl+C something else and then run the above 2 lines again (in shell) with key 'key1' replaced by some other key name, say 'key2'. This is how you can save multiple content to clipboard against different keys (*normally, you can copy only one content on clipboard; once you Ctrl+C something else, previous content of clipboard gets erased*).
 #Now, to get the contents copied to the clipboard, you need to access it key by key as shown below:

```
sys.argv = ['multiClipboard.pyw', 'key1']
exec(open('multiClipboard.pyw').read())
pyperclip.paste() #on shell, this will print the clipboard content stored against key 'key1'

sys.argv = ['multiClipboard.pyw', 'key2']
exec(open('multiClipboard.pyw').read())
pyperclip.paste() #on shell, this will print the clipboard content stored against key 'key2'

sys.argv = ['multiClipboard.pyw', 'list'] #list the keys' names of all clipboard contents
exec(open('multiClipboard.pyw').read())
pyperclip.paste() #o/p: "['key1', 'key2']"
```

ORGANISING FILES

Modules: shutil, send2trash, zipfile, os, re

Functions: copytree(dirpath, copy_dirpath), move(src_path, destn_path), unlink(path), rmdir(path), rmtree(path), send2trash(filename), walk(path), ZipFile(), namelist(), getinfo(filename) -> this provides attributes like 'file_size' and 'compress_size'; extract(filename), extract(filename, to_path), extractall()

The 'shutil' (shell utilities) Module: it lets you copy, move, rename & delete files (and folders) in your progs.

```
import shutil, os
print(os.getcwd())
```

'''-----RUN THE LINES ONE AT A TIME-----'''

'''Copying files & folders'''

#copy(src, destn) function: creates a copy of file from src to destn path

```
##shutil.copy('../Python/FileOps/OrganisingFiles/bacon.txt', '../Python/FileOps/OrganisingFiles/bacon_copy.txt') #copying file to
source location itself
##shutil.copy('../Python/FileOps/OrganisingFiles/bacon.txt', '../Python/FileOps/OrganisingFiles/DemoSubfolder') #copying the file
to other folder location
##shutil.copytree('../Python/FileOps/OrganisingFiles', '../Python/NewBackup') #copies content in 'OrganisingFiles' folder to a
'NewBackup' folder (gets auto-created)
```

'''MOVING & RENAMING FILES & FOLDERS'''

```
##shutil.move('../Python/FileOps/OrganisingFiles', '../Python') #moves 'OrganisingFiles' dir to 'Python' dir
##shutil.move('../Python/OrganisingFiles', '../Python/FileOps') #moves 'OrganisingFiles' dir back into 'FileOps' dir
```

In next 2 lines, 'bacon.txt' is moved to new location and then back to original location

```
##shutil.move('../Python/FileOps/OrganisingFiles/bacon.txt', '../Python/FileOps/OrganisingFiles/DemoSubfolder')
##shutil.move('../Python/FileOps/OrganisingFiles/DemoSubfolder/bacon.txt', '../Python/FileOps/OrganisingFiles')
```

In next 2 lines, file 'bacon' is moved to new location as 'bacon_copy', then moved back to original location as 'bacon'.

```
##shutil.move('../Python/FileOps/OrganisingFiles/bacon.txt', '../Python/FileOps/bacon_copy.txt')
##shutil.move('../Python/FileOps/bacon_copy.txt', '../Python/FileOps/OrganisingFiles/bacon.txt')
```

In above 2 lines, if there was already a 'bacon' file in destn, THAT file would be overwritten by 'bacon' of src.

CAUTION: If 'move' can't find a folder in destn path in existence, then the file in src path is moved and renamed to that non-existent folder's name (w/o any extnsn). This is highly undesirable. So be careful with using the 'move' function.

```
##shutil.move('../Python/FileOps/OrganisingFiles/eggs.txt', '../Python/FileOps/OrganisingFiles/Demo')
##os.remove('../Python/FileOps/OrganisingFiles/Demo')
```

Folders that make up the destn path must already exist, else exception is thrown (e.g. next line).

```
##shutil.move('../Python/FileOps/OrganisingFiles/eggs.txt', '../Python/Test/FileOps/Test2/OrganisingFiles/eggs2.txt')
```

'''PERMANENTLY DELETING FILES AND FOLDERS'''

You can delete a single file or a single empty folder with functions of the os module, but to delete a folder & all its contents, you use 'shutil' module.

```
##import os
##for filename in os.listdir():
##    if filename.endswith('.Rxt'): #BEWARE the extnsn you provide here coz ALL files with this extnsn will be erased permanently
in next line
##        os.unlink(filename) #unlink(path) DELETES the file at 'path'
#instead of running the above code as it is, first comment out the 'unlink' line and add a 'print(filename)' stmt beneath it;
then run to find out & confirm which files will be deleted
```

SAFELY DELETING WITH THE 'send2trash' MODULE:

Python's `shutil.rmtree()` fn irreversibly deletes files and folders which is dangerous. So, third-party 'send2trash' module is used as it sends the files and folders to recycle bin instead of permanently deleting them. Also, the 'send2trash()' fn can only send files to recycle bin; it can't pull files out of it.

```
##import send2trash
##first run the following three lines together
##baconFile = open('../Python/FileOps/OrganisingFiles/bacon_trash.txt', 'a')
##baconFile.write('Bacon is not a vegetable.')
##baconFile.close()
Now, uncomment the following line and run
##send2trash.send2trash('../Python/FileOps/OrganisingFiles/bacon_trash.txt') #file 'bacon_trash' is send to recycle bin
```

WALKING THROUGH A DIRECTORY TREE AND TOUCHING THE FILES/SUBFOLDERS AS YOU GO

```
##import os
#in below for-loop, walk() returns string of current folder's name, & lists of strings for vars 'subfolders' & 'filenames' &
#these vars can have their own for-loops (as in the following code)
##for foldername, subfolders, filenames in os.walk('../Python/FileOps/OrganisingFiles'):
##    print('CURRENT FOLDER IS : '+foldername)
##
##    for subfolder in subfolders: print('SUBFOLDER OF '+foldername+' IS :'+subfolder)
##    for filename in filenames: print('FILE INSIDE '+foldername+' : '+filename)
##    print('-----')
```

COMPRESSING FILES WITH 'zipfile' MODULE'S 'ZipFile()' FUNCTION

```
##import zipfile, os
##os.chdir('../Python/FileOps/OrganisingFiles')
##newZip = zipfile.ZipFile('demo.zip', 'w') #opening in 'w' mode erases prev content of zip file; open in 'a' (append) mode to
#simply add to an existing ZIP file
##newZip.write('spam.txt', compress_type=zipfile.ZIP_DEFLATED)
##newZip.write('eggs.txt', compress_type=zipfile.ZIP_DEFLATED)
##newZip.write('bacon.txt', compress_type=zipfile.ZIP_DEFLATED)
##newZip.close()
##print(newZip.namelist())
```

'''Reading contents of ZIP files'''

```
##os.chdir('../Python/FileOps/OrganisingFiles')
##zippedList = zipfile.ZipFile('zipped.zip') #already created ZIP archive file in cwd
##print(zippedList.namelist()) #'namelist()' returns a list of strings for all files and folders contained in ZIP file
```

'''Getting info about ZIP file's content'''

```
##zippedInfo = zippedList.getinfo('spam.txt') #'getinfo()' fn returns a ZipInfo object about specified single file
##print(zippedInfo.file_size) #size of file specified in above line
##print(zippedInfo.compress_size) #size of ZIP file
```

'''Extracting ALL FILES AT ONCE from ZIP file'''

```
##zippedList.extractall() #extracts all files and folders from a ZIP file to cwd; delete spam.txt before running this line to see
#effect when this line is run
##zippedList.extractall('./Unzip') #Creates new folder 'Unzip' in cwd & extracts all files & folders into it
```

'''Extracting SINGLE FILE AT A TIME from ZIP file'''

```
##zippedList.extract('spam.txt') #'spam.txt' extracted to cwd
##zippedList.extract('spam.txt', './Unzip') #folder 'Unzip' created and 'spam.txt' extracted into it
##zippedList.close()
```

Projects Done: `appendSpamToFilename.py`, `euFilenameDates.py`